06-02 Entry 처리 시스템 분석

개요

Entry는 Solana의 Proof of History (PoH) 시스템의 핵심 구성 요소로, 블록체인의 시간 순서를 보장하는 데이터 구조입니다. 각 Entry는 이전 Entry의 해시, 해시 횟수, 그리고 트랜잭션들을 포함하여 시간의 흐름과 트랜잭션의 순서를 암호학적으로 증명합니다.

Entry 구조체

핵심 Entry 구조

```
#[derive(Serialize, Deserialize, Debug, Default, PartialEq, Eq, Clone)]
pub struct Entry {
/// 이전 Entry ID 이후의 해시 횟수
pub num_hashes: u64,

/// 이전 Entry ID 이후 `num_hashes`번 해시한 SHA-256 해시값
pub hash: Hash,

/// Entry ID가 생성되기 전에 관찰된 트랜잭션들의 순서가 없는 리스트
/// 이전 Entry ID 이전에 관찰되었을 수도 있지만 결정적 해석을 위해
/// 이 리스트로 밀려난 트랜잭션들을 포함할 수 있음
pub transactions: Vec<VersionedTransaction>,
```

```
}
```

Entry 요약 구조

```
pub struct EntrySummary {
pub num_hashes: u64,
pub hash: Hash,
pub num_transactions: u64,
}
```

Entry 타입 구분

```
pub enum EntryType {
Transactions(Vec<SanitizedTransaction>), // 트랜잭션을 포함한 Entry
Tick(Hash), // Tick Entry (트랜잭션 없음)
}
```

Proof of History (PoH) 통합

Poh 구조체

```
pub struct Poh {
pub hash: Hash, // 현재 해시값
```

```
num_hashes: u64, // 현재까지의 해시 횟수

hashes_per_tick: u64, // 틱당 해시 횟수

remaining_hashes: u64, // 다음 틱까지 남은 해시 횟수

tick_number: u64, // 현재 틱 번호

slot_start_time: Instant, // 슬롯 시작 시간

}
```

PohEntry 구조체

```
#[derive(Debug)]

pub struct PohEntry {

pub num_hashes: u64, // 해시 횟수

pub hash: Hash, // 결과 해시값

}
```

Entry 생성 과정

1. 새로운 Entry 생성

```
impl Entry {

pub fn new(prev_hash: &Hash, mut num_hashes: u64, transactions: Vec<Transacti

// 트랜잭션이 있지만 num_hashes가 0이면 1로 설정

if num_hashes == 0 && !transactions.is_empty() {

num_hashes = 1;
```

```
}

let transactions = transactions.into_iter().map(Into::into).collect::<Vec<_>>()
let hash = next_hash(prev_hash, num_hashes, &transactions);
Entry {
num_hashes,
hash,
transactions,
}
}
}
```

2. 해시 계산 과정

```
pub fn next_hash(
start_hash: &Hash,
num_hashes: u64,
transactions: &[VersionedTransaction],
) -> Hash {
if num_hashes == 0 && transactions.is_empty() {
return *start_hash;
}
```

```
let mut poh = Poh::new(*start_hash, None);
poh.hash(num_hashes.saturating_sub(1));
if transactions.is_empty() {
// Tick Entry: 단순히 해시만 수행
poh.tick().unwrap().hash
} else {
// Transaction Entry: 트랜잭션 해시를 믹스인
poh.record(hash_transactions(transactions)).unwrap().hash
}
}
```

3. 트랜잭션 해시 계산

```
pub fn hash_transactions(transactions: &[VersionedTransaction]) -> Hash {
// 트랜잭션 슬라이스의 해시는 서명들만 해시하면 됨
let signatures: Vec<_> = transactions
.iter()
.flat_map(|tx| tx.signatures.iter())
.collect();
let merkle_tree = MerkleTree::new(&signatures);
if let Some(root_hash) = merkle_tree.get_root() {
*root_hash
```

```
} else {
Hash::default()
}
}
```

PoH 시스템 동작 원리

1. PoH 초기화

```
impl Poh {
pub fn new(hash: Hash, hashes_per_tick: Option<u64>) -> Self {
let hashes_per_tick = hashes_per_tick.unwrap_or(LOW_POWER_MODE);
assert!(hashes_per_tick > 1);
Poh {
hash,
num_hashes: 0,
hashes_per_tick,
remaining_hashes: hashes_per_tick,
tick_number: 0,
slot_start_time: Instant::now(),
}
}
}
```

2. 해시 수행

```
impl Poh {
pub fn hash(&mut self, max_num_hashes: u64) -> bool {
let num_hashes = std::cmp::min(self.remaining_hashes - 1, max_num_hashes);

for _ in 0..num_hashes {
self.hash = hash(self.hash.as_ref());
}
self.num_hashes += num_hashes;
self.remaining_hashes -= num_hashes;

assert!(self.remaining_hashes > 0);
self.remaining_hashes == 1 // 다음에 tick()이 필요하면 true 반환
}
}
```

3. 트랜잭션 기록

```
impl Poh {
pub fn record(&mut self, mixin: Hash) -> Option<PohEntry> {
if self.remaining_hashes == 1 {
return None; // 먼저 tick()을 호출해야 함
```

```
}

// 현재 해시와 믹스인 해시를 결합
self.hash = hashv(&[self.hash.as_ref(), mixin.as_ref()]);
let num_hashes = self.num_hashes + 1;
self.num_hashes = 0;
self.remaining_hashes -= 1;

Some(PohEntry {
num_hashes,
hash: self.hash,
})
}
}
```

4. 틱 생성

```
impl Poh {
pub fn tick(&mut self) -> Option<PohEntry> {
self.hash = hash(self.hash.as_ref());
self.num_hashes += 1;
self.remaining_hashes -= 1;
```

```
// 저전력 모드이거나 남은 해시가 없으면 틱 생성
if self.hashes_per_tick != LOW_POWER_MODE && self.remaining_hashes != 0 {
return None;
}

let num_hashes = self.num_hashes;
self.remaining_hashes = self.hashes_per_tick;
self.num_hashes = 0;
self.tick_number += 1;
Some(PohEntry {
num_hashes,
hash: self.hash,
})
}
}
```

Entry 검증 시스템

1. Entry 검증

```
impl Entry {
pub fn verify(&self, start_hash: &Hash) -> bool {
let ref_hash = next_hash(start_hash, self.num_hashes, &self.transactions);
```

```
if self.hash != ref_hash {
warn!(
"next_hash is invalid expected: {:?} actual: {:?}",
self.hash, ref_hash
);
return false;
}
true
}

pub fn is_tick(&self) -> bool {
self.transactions.is_empty()
}
}
```

2. Entry 슬라이스 검증

```
pub trait EntrySlice {
fn verify(&self, start_hash: &Hash) -> bool;
fn verify_cpu(&self, start_hash: &Hash) -> EntryVerificationState;
fn start_verify(&self, start_hash: &Hash, recyclers: VerifyRecyclers) -> EntryV
fn verify_tick_hash_count(&self, tick_hash_count: &mut u64, hashes_per_tick: u6
fn tick_count(&self) -> u64;
```

```
}
```

3. CPU 기반 검증

```
impl EntrySlice for [Entry] {
fn  verify_cpu_generic(&self, start_hash: &Hash) -> EntryVerificationState {
let  genesis = [Entry {
num_hashes: 0,
hash: *start_hash,
transactions: vec![],
}];
let  entry_pairs = genesis.par_iter().chain(self).zip(self);
let  res = PAR_THREAD_POOL.install(|| {
entry_pairs.all(|(x0, x1)| {
let  r = x1.verify(&x0.hash);
if !r {
warn!(
"entry invalid!: x0: {:?}, x1: {:?} num txs: {}",
x0.hash, x1.hash, x1.transactions.len()
);
}
r
})
```

```
});
EntryVerificationState {
verification_status: if  res {
EntryVerificationStatus::Success
} else {
EntryVerificationStatus::Failure
},
poh_duration_us: timing::duration_as_us(&now.elapsed()),
device_verification_data: DeviceVerificationData::Cpu(),
}
}
}
```

4. SIMD 최적화 검증

```
impl EntrySlice for [Entry] {
fn  verify_cpu_x86_simd(&self, start_hash: &Hash, simd_len: usize) -> EntryVerif
// SIMD 길이에 맞춰 정렬
let  aligned_len = ((self.len() + simd_len - 1) / simd_len) * simd_len;
let  mut  hashes_bytes = vec![0u8; HASH_BYTES * aligned_len];
// 해시 데이터 준비
genesis.iter().chain(self).enumerate().for_each(|(i, entry)| {
if  i < self.len() {
```

```
let  start = i * HASH_BYTES;
let  end = start + HASH_BYTES;
hashes_bytes[start..end].copy_from_slice(&entry.hash.to_bytes());
}
});
// SIMD 연산 수행
let  res = PAR_THREAD_POOL.install(|| {
hashes_chunked.par_iter_mut().zip(num_hashes).enumerate().all(|(i, (chunk, num_h
match  simd_len {
8 => unsafe {
(api().unwrap().poh_verify_many_simd_avx2)(
chunk.as_mut_ptr(),
num_hashes.as_ptr(),
);
},
16 => unsafe {
(api().unwrap().poh_verify_many_simd_avx512skx)(
chunk.as_mut_ptr(),
num_hashes.as_ptr(),
);
},
_ => panic!("unsupported simd len: {simd_len}"),
}
```

```
// 결과 검증
self[entry_start..entry_end].iter().enumerate().all(|(j, ref_entry)| {
let start = j * HASH_BYTES;
let end = start + HASH_BYTES;
let hash = Hash::new(&chunk[start..end]);
compare_hashes(hash, ref_entry)
})
})
});
EntryVerificationState { /* ... */ }
}
}
```

트랜잭션 검증 시스템

1. 트랜잭션 검증

```
pub fn verify_transactions(
entries: Vec<Entry>,
verify: Arc<dyn Fn(VersionedTransaction) -> Result<SanitizedTransaction> + Send
) -> Result<Vec<EntryType>> {
PAR_THREAD_POOL.install(|| {
entries.into_par_iter().map(|entry| {
```

```
if entry.transactions.is_empty() {
Ok(EntryType::Tick(entry.hash))
} else {
Ok(EntryType::Transactions(
entry.transactions.into_par_iter()
.map(verify.as_ref())
.collect::<Result<Vec<_>>>()?
))
}
}).collect()
})
}
```

2. GPU 가속 서명 검증

```
pub fn start_verify_transactions(
entries: Vec<Entry>,
skip_verification: bool,
verify_recyclers: VerifyRecyclers,
verify: Arc<dyn Fn(VersionedTransaction, TransactionVerificationMode) -> Result<
) -> Result<EntrySigVerificationState> {
let api = perf_libs::api();
```

```
// GPU 사용 조건 확인
let use_cpu = skip_verification
|| api.is_none()
|| entries.iter().try_fold(0, |accum: usize, entry: &Entry| -> Option<usize> {
if accum.saturating_add(entry.transactions.len()) < 512 {
Some(accum.saturating_add(entry.transactions.len()))
} else {
None
}
}).is_some();

if use_cpu {
start_verify_transactions_cpu(entries, skip_verification, verify)
} else {
start_verify_transactions_gpu(entries, verify_recyclers, verify)
}
}
```

3. GPU 서명 검증 구현

```
fn start_verify_transactions_gpu(
entries: Vec<Entry>,
```

```
verify_recyclers: VerifyRecyclers,

verify: Arc<dyn Fn(VersionedTransaction, TransactionVerificationMode) -> Result<

) -> Result<EntrySigVerificationState> {

// 트랜잭션을 패킷으로 변환

let mut packet_batches = entry_txs

.par_iter()

.chunks(PACKETS_PER_BATCH)

.map(|slice| {

let mut packet_batch = PacketBatch::new_with_recycler(

&verify_recyclers.packet_recycler,

slice.len(),

"entry-sig-verify",

);

// 패킷 데이터 채우기

let res = packet_batch.par_iter_mut().zip(slice.into_par_iter().map(|tx| tx.to_

.all(|(packet, tx)| {

*packet.meta_mut() = Meta::default();

Packet::populate_packet(packet, None, &tx).is_ok()

});

if res { Ok(packet_batch) } else { Err(TransactionError::SanitizeFailure) }

})

.collect::<Result<Vec<_>>>()?;
```

```
// GPU 검증 스레드 시작
let gpu_verify_thread = thread::Builder::new()
.name("solGpuSigVerify".into())
.spawn(move || {
let mut verify_time = Measure::start("sigverify");
sigverify::ed25519_verify(
&mut packet_batches,
&tx_offset_recycler,
&out_recycler,
false,
num_packets,
);
let verified = packet_batches.iter()
.all(|batch| batch.iter().all(|p| !p.meta().discard()));
verify_time.stop();
(verified, verify_time.as_us())
})
.unwrap();


Ok(EntrySigVerificationState {
verification_status: EntryVerificationStatus::Pending,
entries: Some(entries),
device_verification_data: DeviceSigVerificationData::Gpu(GpuSigVerificationData
```

```
thread_h: Some(gpu_verify_thread),
}),
gpu_verify_duration_us: 0,
})
}
```

성능 최적화

1. 병렬 처리

Entry 검증과 트랜잭션 처리는 Rayon을 사용한 병렬 처리로 최적화됩니다:

```
lazy_static! {
static  ref PAR_THREAD_POOL: ThreadPool = rayon::ThreadPoolBuilder::new()
.num_threads(get_max_thread_count())
.thread_name(|i| format!("solEntry{i:02}"))
.build()
.unwrap();
}
```

2. SIMD 최적화

x86 아키텍처에서 AVX2/AVX512 SIMD 명령어를 사용하여 해시 검증을 가속화합니다.

3. GPU 가속

대량의 서명 검증 작업은 GPU를 활용하여 처리 속도를 향상시킵니다.

Entry와 PoH의 시간 보장

1. 시간 순서 보장

```
impl Poh {
pub fn target_poh_time(&self, target_ns_per_tick: u64) -> Instant {
assert!(self.hashes_per_tick > 0);
let offset_tick_ns = target_ns_per_tick * self.tick_number;
let offset_ns = target_ns_per_tick * self.num_hashes / self.hashes_per_tick;
self.slot_start_time + Duration::from_nanos(offset_ns + offset_tick_ns)
}
}
```

2. VDF (Verifiable Delay Function) 특성

- 각 해시는 이전 해시의 결과를 기반으로 계산됨
- 병렬화가 불가능한 순차적 계산
- 시간의 경과를 암호학적으로 증명

3. 해시 성능 측정

```
pub fn compute_hash_time_ns(hashes_sample_size: u64) -> u64 {
let mut v = Hash::default();
```

```
let start = Instant::now();
for _ in 0..hashes_sample_size {
v = hash(v.as_ref());
}
start.elapsed().as_nanos() as u64
}

pub fn compute_hashes_per_tick(duration: Duration, hashes_sample_size: u64) ->
let elapsed_ms = compute_hash_time_ns(hashes_sample_size) / (1000 * 1000);
duration.as_millis() as u64 * hashes_sample_size / elapsed_ms
}
```

정리

Entry 처리 시스템은 Solana의 핵심 혁신인 Proof of History를 구현하는 핵심 컴포넌트입니다:

1. **시간 순서 보장**: 해시 체인을 통한 암호학적 시간 증명
2. **트랜잭션 통합**: 트랜잭션과 시간 정보의 결합
3. **병렬 검증**: CPU, SIMD, GPU를 활용한 고성능 검증
4. **VDF 특성**: 순차적 해시 계산을 통한 시간 지연 증명
5. **확장성**: 대량의 Entry와 트랜잭션 처리 최적화

이러한 설계를 통해 Solana는 높은 처리량과 낮은 지연시간을 달성하면서도 블록체인의 시간 순서를 보장할 수 있습니다.